arXiv:2604.05589v2 [cs.CR] 19 Oct 2026

Foundations for Investigations of Agentic AI Systems from the Forensic Analysis of OpenClaw

JAN GRUBER*, KASTEL Security Research Labs, Karlsruhe Institute of Technology (KIT), Germany
JAN-NICLAS HILGERT, Fraunhofer Institute for Communication, Information Processing and Ergonomics FKIE, Germany

Agentic AI systems are increasingly deployed as personal assistants and virtual employees, and are likely to become a common object of digital investigations and incident response. Yet little is known about how their internal state and actions can be reconstructed during forensic analysis, and systematic forensic approaches remain largely unexplored. We present the first empirical forensic study of a personal AI assistant, with OpenClaw as a case study. We examine its technical design via static code analysis and apply differential forensic analysis to identify recoverable traces across stages of the agent interaction loop. We classify and correlate these traces, structuring the examination around representative investigative questions. We propose a five-plane agent artifact taxonomy that captures recurring investigative patterns; its structural completeness is assessed by cross-mapping against published artifact inventories of two related AI systems. To aid investigations, we release an open-source artifact-extraction tool with a machine-readable schema. Finally, we highlight a foundational challenge: LLM-mediated execution adds both an abstraction layer and substantial nondeterminism to trace generation. Because tool selection and state transitions are co-determined by the model, the execution environment, and the evolving context, identical inputs need not yield identical traces.

CCS Concepts: • **Applied computing** → **Evidence collection, storage and analysis**; **Investigation techniques**; **System forensics**; • **Computing methodologies** → *Intelligent agents*.

Additional Key Words and Phrases: AI forensics, agentic AI, digital investigations, non-traditional forensic scenarios

1 Introduction

AI agents represent a fundamental leap in artificial intelligence, transforming chatbots into autonomous systems that can reason, plan, and execute real-world workflows. The highly popular open-source software project *OpenClaw* democratizes this technology, allowing non-technical users to deploy powerful agents as personal assistants without complex coding, which suggests these systems will soon become common in digital investigations.[1] Despite this trailblazing trend, the forensic community currently lacks both experience with, and an understanding of, how to identify and analyze the traces produced by these systems.

1.1 Motivation

In the past months, we have seen agentic AI rapidly evolve from tools for coding and online workflows into "personal AI assistants" and further into semi-autonomous entities embedded in users' daily lives. While they resemble earlier smart-home assistants in their ability to support the user in accomplishing tasks, for instance in the form of Amazon's ecosystem [7, 8], these new systems extend far beyond passive interaction: they

*Corresponding author

[1] See https://web.archive.org/web/20260219034956/https://www.wired.com/story/clawdbot-moltbot-viral-ai-assistant/, accessed on 15.07.2026.

Authors' Contact Information: Jan Gruber, jan.gruber@kit.edu, KASTEL Security Research Labs, Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany; Jan-Niclas Hilgert, jan-niclas.hilgert@fkie.fraunhofer.de, Fraunhofer Institute for Communication, Information Processing and Ergonomics FKIE, Bonn, Germany.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in *Digital Threats: Research and Practice*, https://doi.org/10.1145/3848419.

execute multi-step workflows, integrate deeply with operating systems and online services, and are granted broad permissions over highly sensitive personal data. Furthermore, these systems are intended for always-on assistance running 24/7 and operate as more or less autonomous entities while their actions can produce real-world effects that are not fully anticipated or even observed by the human operator. Shapira et al. [26] conducted laboratory experiments and observed OpenClaw instances to show highly concerning behavior such as "unauthorized compliance with non-owners, disclosure of sensitive information, execution of destructive system-level actions, [...] and partial system takeover" while creating denial-of-service conditions, consuming resources in an uncontrolled manner. When investigating incidents related to AI agents, the central forensic question is not only what happened on the host, but what the agent actually did versus what the user requested, knew, or intended—an attribution problem that current forensic understanding struggles to resolve.

Consequently, a significant research gap exists in the forensic analysis of agentic assistants, rooted both in limited system understanding and the absence of dedicated investigative methodologies. Existing digital forensic approaches are ill-equipped to handle the combination of cloud-mediated reasoning, local execution, and continuous autonomy. This work addresses this gap by establishing the foundation for the forensic examination of AI assistants.

1.2 Contributions

To our knowledge, this work presents the first forensic analysis of a personal AI assistant, using the trailblazing open-source system *OpenClaw* as a representative case study. Our contributions are threefold:

(1) **Systematic forensic characterization.** Based on static code analysis, differential analysis, and in-depth file system examination, we identify, classify, and correlate recoverable on-disk traces with observed agent behavior. We structure the examination around representative investigative questions and demonstrate how they can be addressed through the locally available artifacts.
(2) **Agent artifact taxonomy.** We synthesize the theoretical agent interaction loop and core architectural components with our empirical findings to propose a five-plane artifact taxonomy that provides a reusable framework for the systematic forensic analysis of agentic AI systems.
(3) **Identification of qualitatively new forensic challenges.** We demonstrate that agentic AI systems produce a fundamentally different class of forensic evidence—most notably large language model (LLM) reasoning traces with no analog in traditional software—and that their LLM-mediated execution introduces nondeterminism and an additional layer of abstraction into trace generation, challenging long-standing forensic assumptions about reproducibility and provenance.

To support practical analysis, we implement a prototype tool that automatically extracts and correlates local artifacts from an OpenClaw installation. Experimental data and source code are made publicly available.[2]

2 Background & Related Work

2.1 Agentic AI and AI Agents

Agentic AI denotes the broader architectural paradigm of systems that can autonomously collect data, reason about problems, and execute self-determined tasks with minimal human intervention [22]. Such systems extend generative AI by adding planning, action, memory, and adaptation capabilities, taking initiative on behalf of users rather than merely responding to prompts [16, 18]. Within this paradigm, an *AI agent* (or *single-agent system*) is a computational entity that perceives its environment, reasons about observations, and executes actions in pursuit of explicit goals with limited human oversight [1, 23]. OpenClaw is such a single-agent system; although it can

[2]See https://github.com/jgru/forensic-analysis-of-openclaw, Commit 98edee7.

spawn ephemeral subagents to handle scoped subtasks, these operate under the primary agent's authority and do not constitute independent autonomous entities as in multi-agent architectures.

Architecturally, AI agents are structured around an LLM as the core reasoning component, extended with mechanisms for planning, memory access, and tool selection [16]. A control loop orchestrates their behavior [15, 16]: the agent perceives inputs, incorporates them into the LLM's context and memory, determines a strategy, and executes it through tool-driven actions (e.g., APIs, CLI tools, or software environments) [18]. It then observes the results to update its context and memory—both short-term (episodic) and persistent (retrieval-augmented)—before the next iteration [23].

2.2 Related Work

Prior work can be broadly grouped into two streams. First, researchers have applied machine learning and AI models to conduct or support digital forensic investigations, traditionally through narrow, task-specific methods [10, for an overview] and increasingly through general-purpose LLMs [24], including explorations using agentic AI frameworks [29].

Second, researchers have examined AI systems themselves as forensic objects—as subjects of investigation, instruments enabling wrongdoing, or targets of attack. Our work contributes to this latter (and comparatively sparse) stream. Schneider and Breitinger [25] examined the attribution and accountability challenges that arise when AI systems act with limited or no real-time human oversight. They highlighted the difficulty of determining whether an AI system merely executed predefined logic or exhibited behavior that could be considered independent or emergent. This distinction is central to forensic analysis of autonomous AI agents, such as OpenClaw. In contrast to prior work that focuses on defining AI forensics, recent work has begun to operationalize AI forensics by proposing concrete sources of forensic evidence within AI systems. In this context, Dragonas et al. [9] present a forensic analysis of the ChatGPT mobile application, demonstrating the feasibility of extracting evidence related to LLM interaction in this mobile application. Chernyshev et al. [6] focused on the LLM itself. They were the first to analyze LLM invocation logs to reconstruct intent, usage patterns, and misuse. As the first forensic analysis of agentic tooling, Walker et al. [28] investigated forensic artifacts when using Microsoft's *AutoGen* multi-agent LLM framework. In their experiments, they identified logs, configuration files, agent communication traces, and execution metadata that enable reconstruction of agent behavior and task workflows. Their findings demonstrated that meaningful forensic analysis is possible in multi-agent LLM environments and emphasized the need for specialized methodologies to support investigations involving collaborative AI systems.

While OpenClaw is an orchestration layer utilizing LLMs for reasoning, these works are related insofar as they deal with identifying internal AI artifacts—such as logs and prompts—as forensic evidence. Our analysis of OpenClaw, however, goes beyond these settings, as OpenClaw operates as an AI assistant with substantially greater autonomy and much deeper integration into the host system, enabling persistent state, direct interaction with local resources, and self-directed task execution, which introduces a broader and more complex forensic surface.

Using the taxonomy proposed by Baggili and Behzadan [3], the present paper is situated in the field of *AI Substrate* and *AI Application Forensics*, contributing to the conceptual and methodological foundations of AI forensics and providing empirical insights into the forensic analysis of agentic AI systems.

2.3 Overview of the Functionality

OpenClaw is a self-hosted personal AI assistant designed to translate natural-language instructions into concrete actions across connected devices and services. At a high level, the system comprises a centralized gateway and a primary agent capable of spawning subagents.

OpenClaw enables continuous interaction with users through existing messaging applications as well as dedicated companion apps for macOS, iOS, and Android. Functionally, OpenClaw combines LLM-based reasoning with device-level execution implemented through extensible skills. These skills enable operations such as sending emails, controlling a web browser, managing calendar appointments, and interacting with the local file system. The integration of persistent memory, natural-language reasoning, direct system interaction, and subagent management distinguishes OpenClaw as an agentic system. Its high-level architecture is visualized in Fig. 1.

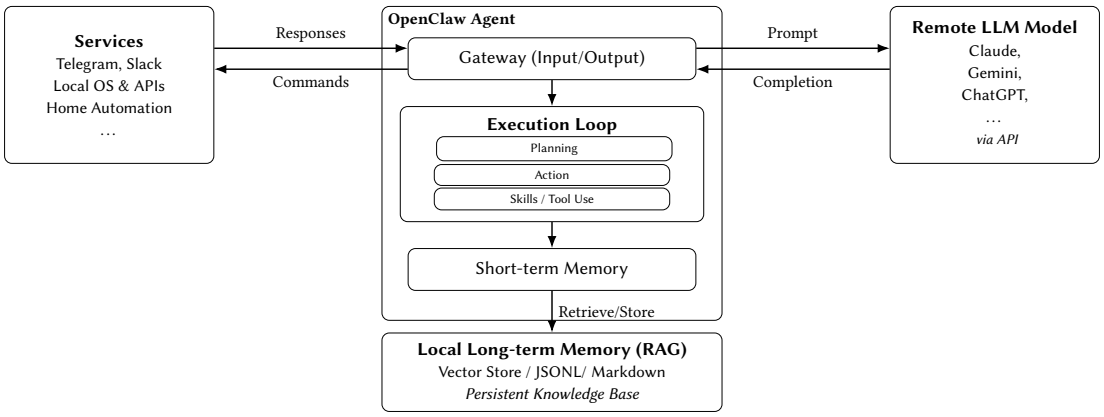


Fig. 1. High-level Architecture of OpenClaw.

From a more technical perspective, OpenClaw is designed as a modular, local-first platform to act as an orchestration layer around an LLM at its core, services through APIs, and data stores (such as file systems). The system is implemented primarily in TypeScript and can be run via JavaScript runtime environments `Node.js` and bun.

2.4 Gateway Service

At the heart of the architecture lies the *gateway*, a Node.js-based service acting as the always-on and remotely accessible central process. It employs an event-driven architecture to handle real-time communication, utilizing WebSockets for client connectivity and internal event emitters for subsystem coordination. The Gateway is responsible for:

- **Lifecycle Management:** Orchestrating the initialization, execution, and termination of sessions.
- **Routing & Dispatch:** Multiplexing incoming messages from external channels (e.g., Telegram, WhatsApp, Slack) to the appropriate agent contexts.
- **Tool Execution:** Providing a runtime environment where agents can invoke local tools, ranging from executing Bash commands to browser automation.

Hence, it is the central component of the event/control-plane.

2.5 Pluggable Communication Channels

OpenClaw abstracts external communication platforms through a modular plugin system. Each channel (e.g., Telegram, WhatsApp via *Baileys*, Discord, Slack) operates as an independent module that normalizes incoming events into a standard internal schema. The architecture decouples the transport layer from the agentic logic. A

"Dock" abstraction manages the connection state (e.g., WebSocket keep-alives or webhook listeners), ensuring that agent cognition remains agnostic to the specific medium of communication.

2.6 Memory and Persistence Layer

An important feature of OpenClaw is its reliance on local persistence for context management. The memory architecture is built upon plain text files and an SQLite database, enhanced with the `sqlite-vec` extension to support vector embeddings. The system maintains several Markdown files where the agent stores important facts to load as context and a `memory.sqlite` containing chunked text and associated vector embeddings in a `chunks_vec_*` table. This allows agents to perform semantic search and retrieval-augmented generation.

2.7 Security and Integrity

To provide a secure environment, OpenClaw relies on the concepts of permissions, containment, and human oversight to mitigate the risk of an autonomous agent executing destructive commands. To do so, the architecture supports a strict *Policy Enforcement Model*. Sensitive operations, particularly those involving shell execution, can potentially be sandboxed using Docker containers with minimal privileges (read-only root, network isolation). Additionally, an *Execution Approval Manager* gates critical actions, ensuring a "Human-in-the-Loop" workflow by holding high-risk tool calls in a pending state until explicit authorization is granted.

3 Methodology

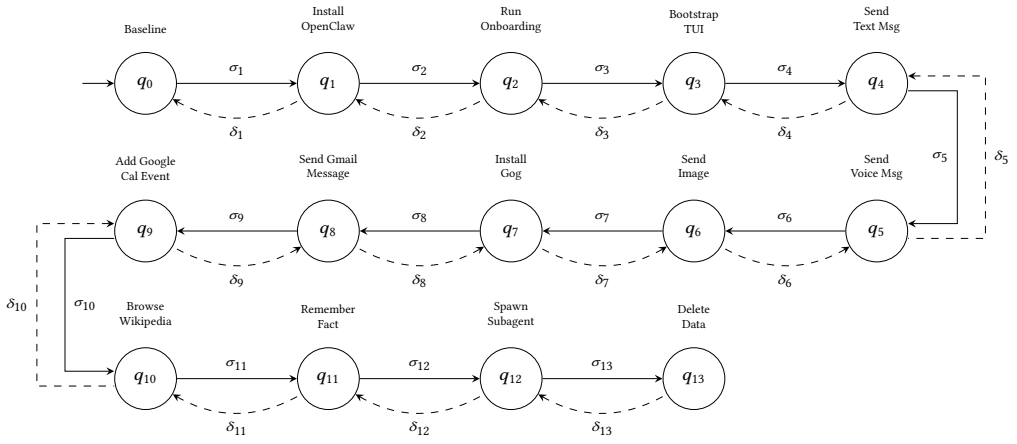


Fig. 2. Data generation workflow. Actions σ_i transition the system states q_i, enabling differential analysis δ_i to isolate artifacts.

3.1 Experimental Design

The experiments were conducted in a virtualized environment managed by Oracle VirtualBox, the guest virtual machine operated on *Debian GNU/Linux 13*. Within this virtualized environment, *OpenClaw* in version `2026.2.2-3`[3] was installed and operated to generate the requisite test data. We chose Linux as the experimental platform

[3]See https://www.npmjs.com/package/openclaw/v/2026.2.2-3, accessed on 15.07.2026.

because OpenClaw is commonly run on cloud VMs and the recommended way to install the software on Windows is using WSL2, making Debian-based Linux a representative target.

Following the methodology of Kälber et al. [17], we systematically generated test data by executing a series of actions ($\sigma_i \in \Sigma$) that encompass installation, setup, and various interactions with the agent. Each action transitions the system to a distinct state (q_i), which is captured as a disk image, starting from a baseline q_0. Fig. 2 illustrates this process, enabling the isolation of artifacts tied to specific agent functionalities.

3.2 Analysis Procedure

Our analysis combined static code analysis, differential analysis, and file inspection to locate and interpret relevant traces. Differential forensic analysis is a well-established method [14, 17] for systematically cataloging application-specific artifacts under controlled conditions. Crucially, the artifact catalog produced by this approach is not bound to the laboratory setting in which it was derived: once the locations, formats, and semantics of relevant traces are known, practitioners can locate and interpret them. First, we performed static code analysis of the source code. This step provided an overview of the application's architecture, its components and their main functionality, and potential data storage locations (such as database schemas or configuration files) before generating any user data.

Next, we applied the differential forensic analysis [14]. This allowed us to isolate the specific effects of an action by comparing the differences between two sequential images. We utilized The Sleuth Kit in conjunction with the `DFXML` library [13] and the Python script `idifference2.py` [12], to programmatically identify changes δ_i in the file system induced by the action σ_i in question by comparing two consecutive state images $\langle q_i, q_{i+1} \rangle$. The tool detects changes across five specific categories: file or directory creation, deletion, renaming, content modification, and timestamp updates. To focus the investigation, we applied a filter to exclude system background noise unrelated to the executed actions.

Finally, the remaining candidates of file system changes were subsequently verified and inspected in-depth. We parsed the relevant files—including plain text files, SQLite databases, JSONL files, and others—using standard forensic tooling and custom scripts. The data recovered from these files were then compared against the known inputs from the test data generation phase to confirm the artifacts' content.

4 Forensic Analysis of OpenClaw

We now present the results of our analysis. We begin with an overview of the main artifact locations, then structure our findings around the core components of the agent architecture described in Section 2.1: the agent's cognitive engine and configured identity, its accumulated knowledge, its communication interfaces, and its executed actions. This organization reflects the modular design of agentic AI systems, where each architectural component produces forensically distinct artifacts that require different analytical approaches. For each component, we address representative investigative questions and introduce the relevant artifacts in detail.

For reproducibility and documentation, we provide the description of the experiments in literate programming style, experimental data, and a tool supporting the analysis as supplementary material (Section A).[4] There, we documented our main findings in the artifacts YAML format to feed machine-readable knowledge bases, like the *Digital Forensics Artifacts Repository* [20].

4.1 Artifact Overview

While standard Linux OS artifacts are well understood and should of course be included in an investigation, we focus in the following on artifacts specific to OpenClaw. In a standard installation, the vast majority of data linked to the AI agent resides in a hidden directory named `.openclaw` in the user's home directory. Table 1 summarizes

[4]See https://github.com/jgru/forensic-analysis-of-openclaw.

Table 1. Overview of OpenClaw artifact locations and their forensic relevance. Phase columns indicate relevance to: **Char.** = Agent Characterization, **Know.** = Knowledge Reconstruction, **Inter.** = Interaction Tracing, **Act.** = Action Reconstruction. Relevance levels: ● = primary evidence, ◐ = secondary evidence, blank = not relevant.

Path	Contains	Char.	Know.	Inter.	Act.
`openclaw.json`	Agent, model, channel, and permission settings	●		●	
`openclaw.json.bak*`	Historical configuration snapshots	●		◐	
`credentials/`	Channel authentication (Telegram, WhatsApp)	◐		●	
`agents/{id}/agent/auth-profiles.json`	Provider credentials (OAuth tokens, API keys)	●			
`devices/`, `identity/`	Paired companion devices and keys			●	
`workspace/*.md`	Agent persona, user profile, tool definitions, curated memory	●	●		
`workspace/skills/`	Custom agent skill definitions	●	◐		
`workspace/memory/`	Daily memory logs (created on demand)		●	◐	
`memory/{agent_id}.sqlite`	Semantic search embeddings		●		
`agents/{id}/sessions/sessions.json`	Session metadata and mappings	●	◐	●	◐
`agents/{id}/sessions/*.jsonl`	Full conversation and tool execution history	◐	●	●	●
`media/inbound/`	User-uploaded files (images, voice notes)		◐	●	
`cron/jobs.json`	Scheduled autonomous task definitions	●			●
`cron/runs/{jobId}.jsonl`	Execution logs for scheduled tasks				●
`subagents/runs.json`	Delegated task tracking				●
`/tmp/openclaw/*.log`	Runtime events	◐		◐	●

the primary artifact locations. All paths are relative to this directory unless stated otherwise. Ephemeral runtime logs are written outside this directory to `/tmp/openclaw/*.log`.

4.1.1 Session Transcripts. Session transcripts are the single richest forensic artifact in the system, as they record not only communication but also reasoning traces, tool invocations, and context state. Because they span all aspects of the agent's operation, we introduce them here before turning to the component-specific analysis.

OpenClaw organizes conversations using a two-level hierarchy. A *session* represents a long-lived conversation context and is identified by two complementary keys: (i) a human-readable `sessionKey` (e.g., `agent:main:main`) used for routing, and (ii) a UUID-based `sessionId` that uniquely identifies a session instance. Sessions persist across interactions and record metadata such as the working directory, model/provider overrides, the configured thinking level, and the path to the transcript file. A session transitions through the states `idle` (awaiting input), `processing` (actively executing), and `waiting` (queued messages pending).

Within a session, individual user interactions are executed as *runs*. A run is a transient execution instance identified by a `runId`. Run progression can be traced through debug log messages that mark stages such as `prompt start`, `agent start`, tool execution, `agent end`, and `prompt end`. In parallel, a structured event system emits events on categorized streams: the `lifecycle` stream tracks run state via phases `start`, `end`, and `error`; the `tool` stream records tool invocations (`start`, `update`, `result`); and the `assistant` stream carries model output. Together, these two trace layers enable fine-grained forensic reconstruction of individual runs.

Session Index and Mapping. Local session data is stored under `~/.openclaw/agents/<agentId>/sessions/`. Within this directory, `sessions.json` tracks current and past sessions and maps `sessionKey` values to their corresponding `sessionId` values. The `sessionId` can then be used to locate the session transcript file `<sessionId>.jsonl`.

Session Transcript Files. Session history is persisted in JSONL files located at `~/.openclaw/agents/-<agentId>/sessions/<sessionId>.jsonl`, where each line is a self-contained JSON object with `id` and `parentId` fields. The file begins with a `session` header entry (version 3) containing the session UUID, creation timestamp, and working directory. Subsequent entries record configuration changes, extension-injected

state, and—most importantly—the conversation itself as `message` entries carrying a `role` field and typed content blocks. The specific record types are described alongside the components they relate to in the following subsections.

Lifecycle and Recovery. OpenClaw employs a soft-delete mechanism: when a session is deleted, the transcript is renamed in place with a `.deleted.<timestamp>` suffix and removed from the `sessions.json` index, rather than being expunged.

4.1.2 Log Data. Log data is stored in `/tmp/openclaw/`. The directory may contain multiple log files, typically one per day, named `openclaw-YYYY-MM-DD.log`. The retention policy is hard-coded: log files older than 24 hours are deleted. The log cleanup routine runs automatically when the logger is first initialized (early in application startup) or when the logger configuration changes.

Log data is stored in JSON format, with each line representing an independent JSON object that includes details about the subsystem that generated the entry, along with a `time` field that records when it was emitted.

4.2 Agent Characterization

The first step in examining an agentic system is to establish what the agent is, how it reasons, and what it could do. This requires answering several investigative questions: How is the agent configured? What LLM drives its reasoning? What capabilities and permissions are available? How has the configuration changed over time?

4.2.1 Configuration and Identity. The primary configuration is stored in `openclaw.json` under the `agents` section. It defines defaults such as the selected model (e.g., Gemini 2.5 Flash), as well as global settings including the workspace root, which defaults to `workspace/` within `~/.openclaw/`. During onboarding, OpenClaw creates an initial agent named `main`; additional agents can be added later and are listed in the `list` entry. Each agent entry specifies, among other parameters, the model, workspace path, and an agent directory.

The agent's persona and operational boundaries are defined by a set of workspace files initiated by `AGENTS.md` and `BOOTSTRAP.md` (the latter is deleted after initialization by the agent itself). Crucial files include: `IDENTITY.md` (agent name/avatar/"creature type"), `SOUL.md` (core truths, personality traits, and safety boundaries), `TOOLS.md` (environment-specific technical details, e.g., SSH hosts and aliases), and `HEARTBEAT.md` (periodic awareness tasks executed in the main session). These files, together with the system prompt assembled from them, constitute the agent's configured identity and behavioral policy.

Agent-specific provider credentials (e.g., API keys or OAuth tokens) are stored in `auth-profiles.json` within the respective agent directory. Session-level configuration and operational metadata can be reconstructed from `agents/{id}/sessions/sessions.json`, which may record per-session parameters such as the active provider/model as well as usage statistics (e.g., token utilization).

4.2.2 Capabilities and Permissions. OpenClaw distinguishes between *tools* and *skills*. Tools represent executable actions (e.g., reading/writing files or invoking `exec` to run commands), whereas skills provide task-specific guidance and prompt material to teach the agent how to use tools effectively.

Skills are defined by a `SKILL.md` file and can be stored either globally in `skills/` or agent-locally under `workspace/skills/`. In addition, OpenClaw may provide bundled skills (e.g., guidance for querying weather information). Tool implementations reside in the OpenClaw codebase (e.g., `src/agents/tools/*.ts`).

Evidence of tool and skill availability can be extracted from an agent's `sessions.json`. For skills, the `skillsSnapshot` structure may include (i) a list of skills, (ii) `resolvedSkills` with the resolved on-disk path to each `SKILL.md` and its `source` (e.g., `openclaw-bundled`), and (iii) the generated prompt text that was provided to the model. In practice, we observed that the `systemPromptReport` prompt-reporting structure may not persist reliably over time. When present, it can additionally summarize exposed tools by name and provide

schema/summary length metrics; however, it does not preserve the full tool descriptions as transmitted to the model, nor does it document the concrete implementation behind an exposed tool. The same structure also records injected workspace files, as discussed in Section 4.3.2.

Scheduled automation capabilities are defined in `cron/jobs.json`; each job captures the schedule (one-shot `at`, fixed-interval `every`, or `cron` expression with optional timezone) and how it is executed, along with payload and runtime state. These definitions reveal what autonomous actions the agent was *configured* to perform. Run history for these jobs is analyzed in Section 4.5.

4.2.3 LLM Core and Reasoning. A distinguishing feature of agentic AI systems is that the agent's decision-making is mediated by an LLM whose reasoning process produces forensically recoverable traces. These artifacts have no direct analog in the forensic analysis of traditional, rule-based software. Assistant messages in the transcript additionally record usage metadata—token counts, cost breakdowns, `stopReason` (e.g., `stop` or `toolUse`), and provider-specific data—enabling investigators to quantify model utilization per interaction.

The LLM model identity—including provider, model version, and thinking level—is recorded in `openclaw.json` and per session in `sessions.json`. Mid-session model changes are captured as `model_change` events in the session transcript. Reasoning traces appear as `thinking` content blocks within assistant messages. These chain-of-thought blocks may reveal how the agent interpreted a request, which alternatives it considered, and why it selected a particular tool.

The agent layer here is not the source of these traces—it only records what it receives. OpenClaw persists whatever the model provider returns through its API, and the content, granularity, and even the presence of reasoning traces are determined upstream. We observed that it depends on the configured model whether a full chain-of-thought as plaintext alongside the final answer is included or not. For instance, providers of current flagship models like Claude or Gemini have chosen to not expose verbatim thoughts but rather summaries and *thought signatures*, described as "encrypted representation of the model's internal reasoning state".[5] Hence, in these cases, investigators examining an agent installation are not analyzing the reasoning of the agent in any direct sense; they are analyzing the partial view of that reasoning that the provider chose to emit at the time of the interaction. Nevertheless, for the model *Gemini 2.5 Flash*, we found the thought summaries to be insightful and comprehensive.

4.2.4 Historical Configuration. Historical configurations can be recovered from the configuration backup files (e.g., `openclaw.json.bak`, `openclaw.json.bak.1`, ...). These snapshots preserve prior configurations alongside metadata such as `meta.lastTouchedAt` and `meta.lastTouchedVersion`, enabling approximate point-in-time reconstruction: the configuration "in effect" for a given time window can be approximated by selecting the most recent backup whose `lastTouchedAt` precedes that time. Differencing consecutive snapshots further reveals what changed.

Because the effective system prompt and capability envelope may not be stored persistently, investigators should consider complementary sources such as *log data*. In our analysis, logs contained specific `google tool schema snapshot` entries that enumerate the tools exposed to the model for a run. While these entries are not explicitly linked to a particular session, they are typically emitted immediately after a run starts and can therefore often be associated with the corresponding session via temporal proximity. As logs are continuous, such snapshots can help identify capability changes over time and detect orphaned tools that were previously available but have since been removed. Analysts should note that snapshots may be recorded in provider-specific schema formats (e.g., Google tool schema), which can vary across model providers.

[5] See https://ai.google.dev/gemini-api/docs/thinking#signatures, accessed on 15.07.2026.

4.3 Knowledge Reconstruction

Having established the agent's identity, reasoning engine, and capabilities, the investigator next reconstructs what the agent knew. This addresses questions such as: What information had the agent accumulated? What was in its episodic and long-term memory? What context was available to the LLM at the time of the events under investigation?

Because an agent's behavior is shaped not only by its configuration and user input but also by its evolving memory, loaded workspace files, and dynamically available tools, the same request can produce different outcomes at different points in time. The goal is therefore to approximate, as closely as the available evidence allows, the knowledge state that was active during the relevant period.

4.3.1 Episodic and Long-term Memory. OpenClaw implements memory primarily as plain Markdown files in the agent workspace. `workspace/memory/YYYY-MM-DD.md` serves as an append-only daily log; updates to this file are triggered by user requests such as "remember this". At session start, the agent is instructed (via `AGENTS.md`) to read the two most recent daily memory files. The curated `MEMORY.md` is injected into the system prompt by the platform.

OpenClaw can additionally build a vector index over memory files to support semantic retrieval, stored in `memory/<agent_id>.sqlite`.

4.3.2 Contextual Knowledge at Incident Time. A central challenge is reconstructing what the agent knew *at a specific point in time*. The `USER.md` file, for instance, contains learned user details such as timezone, active projects, and preferences—information that accumulates and evolves as the agent interacts with the user.

When present, the `systemPromptReport` in `sessions.json` (cf. Section 4.2) may include an `injectedWorkspaceFiles` list (with `name`, `path`, and injected character counts), which helps approximate which workspace and memory files were injected into the model context. However, this structure may not persist reliably, may be missing entirely, or may only reflect the latest recorded state rather than every individual interaction.

One complementary approach is to reconstruct historical versions of knowledge files from the session transcript itself. When the agent uses file-manipulation tools such as `read`, `write`, or `edit`, the session JSONL records the invoked operation and its payload. This allows investigators to recover prior file contents as last observed or modified by the agent and to build an approximate timeline of context evolution driven by the LLM. However, this approach cannot account for *external* modifications to injected files that occur outside the agent's tool interface (e.g., direct user edits in the workspace).

For long-running sessions, investigators should also pay attention to `compaction` entries in the transcript. These indicate where the context window was summarized and mark locations where the agent's context may have changed.

4.4 Interaction Tracing

The investigator next reconstructs the communication between users and the agent. This requires answering: Through which channels could a user interact with the agent? What and when did the user communicate? What information was visible to the user at any given point?

Together, these findings establish the human side of the interaction and define the boundary between what was requested and what the user could observe.

4.4.1 Communication Channels. Channel configuration is centralized in the `channels` section of `openclaw.json`. It defines, for each supported channel (e.g., Telegram, WhatsApp, Discord), whether the channel is enabled and which access policies apply.

Because OpenClaw uses channel-specific implementations—for example, the Telegram Bot API for Telegram and the Baileys library for WhatsApp—the resulting artifacts and their storage locations are likewise channel-specific. For Telegram, the bot token is stored directly in `openclaw.json`, while `credentials/telegram-allowFrom.json` stores the `allowFrom` entry used for permission control. For WhatsApp, credentials and auxiliary material (e.g., pre-keys) are stored under `credentials/whatsapp/default/`.

In addition, `agents/<id>/sessions/sessions.json` can contain per-session channel metadata, such as the current and last `channel` as well as the last recipient identifier (e.g., `telegram:id`). `sessions.json` may optionally record an origin entry that stores the channel in `provider` and carries channel-specific information, e.g., sender identifier in the `from` field or a conversation `label`, which can be correlated to user messages in a transcript file. Log entries from subsystems such as `gateway` or channel-specific subsystems (e.g., `gateway/channels/telegram`) document the connection state and operational health of communication channels over time. Furthermore, comparing configuration backups enables approximate historical reconstruction of channel enablement and policy changes.

4.4.2 Message Recovery and Media. Within the session JSONL files, user-to-agent messages are indicated by `role: user` and include the full text content as well as a timestamped event record.

For some channels, additional attribution is embedded in message headers. For example, Telegram messages include a structured header inside the stored user message content (e.g., `[Telegram <name> (@<handle>) id:<id> <YYYY-MM-DD> <HH:MM> UTC]`). When present, it can be parsed to extract channel attribution, sender identity (e.g., display name, username, platform-specific user ID), and message timestamp. A platform message identifier appears on a separate line as `[message_id: N]`.

When a user sends media attachments, OpenClaw stores inbound files under `~/.openclaw/media/inbound/` using one of two filename formats: if the channel provides an original filename, it becomes `{sanitized_original}—{uuid}.{ext}`; otherwise, only the UUID is used. The extension is determined by MIME type detection. Corresponding session events include a note in the format `[media attached: PATH (mime/type) | URL]`, allowing media files to be linked back to the conversational timeline.

4.4.3 Visibility to the User. The transcript also contains assistant-side events (`role: assistant`) that may contain multiple content types (e.g., tool-related messages, intermediate reasoning, and user-facing text). According to the implementation, for providers that use reasoning tags (e.g., Google Gemini), only content of type `text` wrapped in `<final>` tags is forwarded to the user, while internal thinking blocks (wrapped in `<think>` tags) and tool execution details are not displayed. For providers with native thinking support, this filtering is not enforced. Accordingly, forensic reconstruction should distinguish between the full internal transcript stored on disk and the subset of content that was actually observable to the user.

4.5 Action Reconstruction

This phase focuses on the actions the agent performed. The investigator constructs a timeline of tool invocations, file operations, network requests, scheduled tasks, and other executed actions. Key questions include: What is the timeline of executed actions? Did the agent execute unprompted or autonomous actions?

The output of this phase is a factual record of what the agent did and when, without yet attempting to explain why.

4.5.1 Tool Invocations and Effects. Tool executions are recorded as paired events in the session JSONL files: `toolCall` content blocks in assistant messages specify the tool name, a unique call identifier (`id`), and the arguments; subsequent `role: toolResult` messages reference this identifier as `toolCallId` and contain the output in `content`, an `isError` flag, and a `timestamp`. For shell commands (`exec` tool), an optional `details` object may include `durationMs`, `exitCode`, and `status`.

Log files are an additional source as they contain specific entries for tool start and end that include `runId` and `toolCallId`, enabling cross-source correlation. Anti-forensics indicators include tool calls present in logs but absent from the session transcript, which can suggest session-file tampering.

4.5.2 Autonomous and Scheduled Actions. Run history for scheduled tasks is appended to `cron/runs/<jobId>.jsonl` with timestamps and outcomes, enabling investigators to flag activity that lacks an immediately preceding user request. Each job may be executed either in the main session via an injected system event or in an isolated `cron:<jobId>` run; this distinction determines whether cron-triggered actions appear in the main session transcript or in separate session files.

In addition, the agent may autonomously delegate work to subagents. Delegation can be reconstructed from `sessions_spawn` tool calls (task prompt, child session ID, cleanup policy, token use, and result summary) and from `sessions.json`, which may record the parent in `spawnedBy` and reference the child under keys such as `agent:main:subagent:<uuid>`. If the cleanup policy deletes the entry from `sessions.json`, the child session transcript often remains on disk, allowing investigators to link parent and child via the recorded session identifiers and to attribute seemingly user-initiated events (`role: user`) in the subagent transcript to agent-driven spawning.

OpenClaw additionally maintains a lightweight registry of spawned subagent runs in `subagents/runs.json`. It records high-level metadata such as which subagent was spawned for which task and when it started and ended, which can be useful when session records are incomplete. Entries are created when a subagent is spawned and are updated as lifecycle events occur; depending on the configured cleanup policy, they may be removed immediately after completion or retained only temporarily before being swept once their `archiveAtMs` deadline passes (defaulting to 60 minutes after creation), so their presence (or absence) can itself be an indicator of how the system was configured and operated.

4.6 Attribution Analysis

The preceding phases recover evidence about what the agent was, how it reasoned, what it knew, how it communicated, and what it did. This final phase synthesizes evidence *across* all areas to address questions of causality and responsibility: Can specific actions be linked to specific user requests? What degree of autonomy was exhibited? What are the limits of the reconstruction?

Unlike the evidence-recovery phases, attribution is an *interpretive* activity: it does not recover new artifacts but reasons about the relationships between artifacts already identified.

4.6.1 Linking Actions to Origins. The investigator traces each significant action backward through the session transcript to determine its origin. Did the reasoning trace cite a user message? A memory entry? A cron trigger? Its own prior reasoning? The chain from user request through agent reasoning to tool invocation can often be reconstructed from the sequence of `role: user`, `role: assistant` (with `thinking` blocks), and `toolCall`/`toolResult` entries within a session.

4.6.2 Autonomy Assessment. Indicators of autonomy are distributed across multiple artifact sources. Cron run history quantifies scheduled actions that occurred without a preceding user message. Subagent delegation reveals autonomous task decomposition. Each observed action can be classified on a spectrum: directly instructed by the user, interpretively derived from an ambiguous instruction, autonomously initiated by the agent, or indeterminate. This classification draws on evidence from all preceding phases—the agent's configuration and reasoning capabilities, its knowledge context, the communication record, and the action timeline.

4.6.3 Reconstruction Boundaries. Several factors limit the completeness of any forensic reconstruction of an agentic AI system. First, the LLM's reasoning is only partially observable through thinking traces. Second, the actual context window assembled at inference time is not stored persistently and memory as well as configuration

may have changed between the incident and the examination. Lastly, as we will discuss in Section 6, there is an inherent nondeterminism of LLM-based decision-making, so even a complete reconstruction of inputs cannot guarantee a unique explanation for the agent's behavior.

5 A Forensic Taxonomy for Agentic AI

AI agents are notably different from regular rule-based software systems, so there is a need for the research community to provide guidance for analyzing agent-specific components such as context, configuration, execution, LLM interaction, and autonomous scheduling. To address this gap, we structure the findings of our OpenClaw analysis into an *Agent Artifact Taxonomy*—a functional classification of five artifact planes, each corresponding to a distinct aspect of the agent's architecture. The taxonomy was developed through a combination of literature review on agentic system architectures, code analysis as well as differential analysis, and validated against the abstract agent interaction loop described in Section 2.1.

5.1 Five Artifact Planes

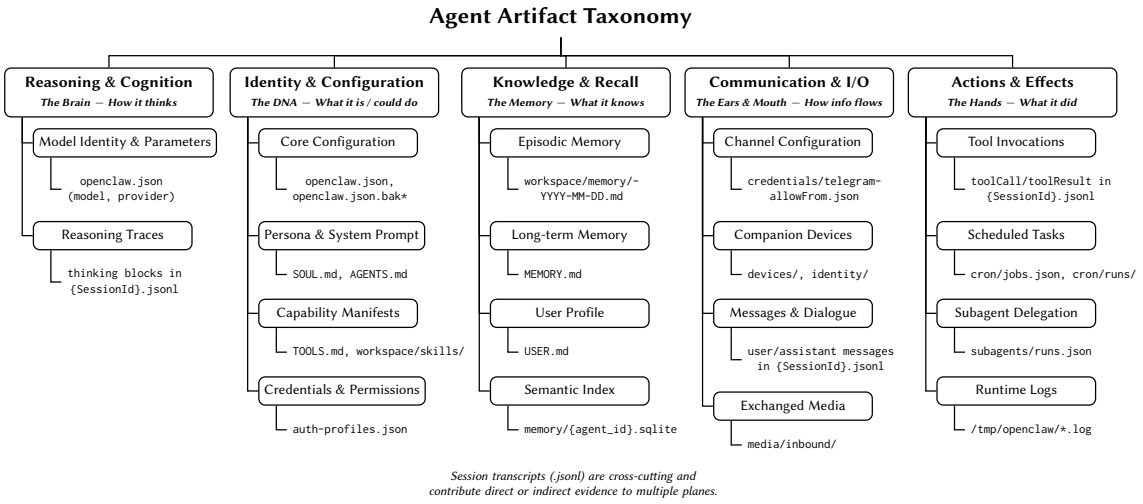


Fig. 3. Agent Artifact Taxonomy for forensic analysis of agentic AI systems. The taxonomy organizes agent-related evidence into five planes, each corresponding to a distinct aspect of the agent's architecture. Session transcripts are cross-cutting and contribute direct or indirect evidence to multiple planes. Representative OpenClaw artifacts are listed as examples.

Just as one might examine attributes indicating a suspect's intent with the cognitive processes, inherent characteristics, knowledge, communications, and actions, the five planes capture analogous aspects of an AI agent. Figure 3 provides an overview with representative artifacts; we briefly introduce each plane below.

1. **Reasoning & Cognition ("The Brain")**: *how the agent thinks.* Captures the LLM's reasoning traces: a category of forensic evidence with no analog in traditional software and the closest approximation to reconstructing agent *intent*.
2. **Identity & Configuration ("The DNA")**: *what the agent is and what it could do.* Encompasses the predominantly *static* artifacts that define the agent's configured identity, capabilities, and operational boundaries.

3. **Knowledge & Recall ("The Memory")**: *what the agent knows.* Covers accumulated knowledge such as episodic memory, user profiles, and semantic indices, distinguished from DNA by being *accumulated at runtime*, often by the agent itself.
4. **Communication & I/O ("The Ears & Mouth")**: *how information flows to and from the agent.* Establishes who interacted with the agent, through which channels, and what each party could observe.
5. **Actions & Effects ("The Hands")**: *what the agent did.* Records concrete actions and their observable effects, providing the factual basis for determining what changes the agent caused.

The planes represent analytical perspectives rather than isolated subsystems, and individual artifacts may participate in multiple planes depending on investigative context. In particular, session transcripts are inherently multi-planar: a single JSONL file directly records reasoning traces (Brain), user and agent messages (Ears & Mouth), and tool invocations (Hands), while additionally containing indirect evidence for the DNA and Memory planes.

5.2 Preliminary Cross-System Assessment

The analytical phases applied in Section 4 map closely to these planes, reflecting the modular architecture of agentic AI systems. While the specific artifacts differ across systems, the taxonomy is designed to generalize: any agent that combines an LLM core with persistent configuration, memory, communication interfaces, and tool execution will produce artifacts classifiable into these five planes. For a preliminary validation, we mapped the taxonomy's five planes against artifacts reported by Walker et al. [28] for the agentic framework AutoGen and by Dragonas et al. [9] for the non-agentic ChatGPT mobile application. All AI-specific artifacts documented in both studies fit within the existing planes without requiring additions, supporting the taxonomy's structural completeness detailed in Table 2. The mapping reveals a discriminative pattern: the Reasoning & Cognition and Actions & Effects planes are both effectively empty for the non-agentic ChatGPT, while the Identity, Knowledge, and Communication planes are populated—mirroring the architectural distinction between systems that merely *use* an LLM and those that *reason and act* via one. AutoGen, despite being a genuine multi-agent framework, shows a similar gap. Grounded in traditional disk, memory, and network forensics, Walker et al. [28] did not consider reasoning traces or structured action records as evidence categories; combined with the reported deprecation of runtime logging since AutoGen v0.2, this left both the Reasoning and Actions planes without recoverable artifacts. This demonstrates how agent-specific evidence classes can go unexamined without a prescriptive taxonomy that defines which artifact categories investigators must consider—ensuring that fundamentally new classes such as reasoning traces are not overlooked.

6 Discussion

6.1 Practical Implications of Our Analysis

AI agents expand the attack and impact surface by mediating high-privilege actions (e.g., file system access, process execution, and network requests) through tool abstractions exposed via asynchronous chat interfaces. Furthermore, the intended autonomy—while beneficial for usability—can increase both the likelihood and severity of unintended or adversarial actions, including data exfiltration, credential misuse, and unsafe system modifications, through adversarial content designed to interfere with visiting agents [11] or compromised agent instances. This interaction model weakens real-time human oversight: tool calls may execute without immediate user confirmation, and their side effects remain unnoticed. The autonomous nature introduces a covert communication risk that differs from supervised interactive sessions. This shifts the trust model from visible session oversight to post-hoc audit and enables "silent informant" behavior: an assistant completes a benign task while covertly transmitting underlying data.

Table 2. Literature-based cross-system mapping of the Agent Artifact Taxonomy. ChatGPT mobile is a cloud-native LLM chat interface without agent capabilities per Section 2.1; it is included as an architectural contrast to illustrate the taxonomy's discriminative scope.

Taxonomy Plane	OpenClaw (this work)	AutoGen (Walker et al. [28])	ChatGPT Mobile (Dragonas et al. [9])
Reasoning & Cognition	`thinking` blocks and `model_change` events in session JSONL; reasoning traces with no analog in traditional software	No reasoning traces recovered or investigated; transient LLM response fragments in volatile memory but recovery inconsistent across tasks; no chain-of-thought, planning step, or inference parameter artifacts	No local reasoning traces (all inference possibly server-side); GPT model version recorded in conversation metadata but contains no chain-of-thought or inference parameters
Identity & Configuration	`openclaw.json`, persona files (`SOUL.md`, `IDENTITY.md`), provider credentials in `auth-profiles.json`	Agent definitions, system prompt, model configuration (`llm_config`), and code execution settings (`work_dir`) in Python source scripts; `pyautogen` installation footprint in `site-packages`; API credentials not recovered	App preferences (BPLIST on iOS, Protobuf on Android), account/user/device/workspace identifiers, subscription plan, custom instructions, authentication tokens
Knowledge & Recall	`memory.sqlite` with vector embeddings, curated `MEMORY.md`, daily memory logs, `USER.md`	Conversational context session-scoped and volatile only (not persisted to disk); logging deprecated since v0.2; no persistent memory, no user profiles, no semantic indices	Conversation SQLite database (Android) and JSON files (iOS); cloud-synced chat history; no persistent agent memory; no local semantic index
Communication & I/O	Multi-channel transcripts (Telegram, WhatsApp), `media/inbound/` attachments, channel config in `openclaw.json`	TLS-encrypted PCAP traces of API connections to OpenAI endpoints; inter-agent dialogue (UserProxyAgent ↔ AssistantAgent) not persistently logged; message content encrypted under TLS 1.3 (decryption not attempted)	Chat messages in local database and JSON; voice recordings (`.m4a`); text-to-speech narrations; uploaded images and files; cloud export archive
Actions & Effects	`toolCall`/`toolResult` pairs in session JSONL, `cron/jobs.json` and `cron/runs/` logs, `subagents/runs.json`	No structured tool invocation logs or agent delegation records recovered; runtime logging deprecated since v0.2	Minimal: DALL-E image generation among tested actions but no dedicated artifact beyond conversation record; no local tool execution, no scheduled tasks, no autonomous actions

In practice, the artifact catalog and taxonomy derived from our controlled analysis directly support field examinations. The taxonomy's five planes provide a structured checklist of evidence categories, while the specific file paths, data formats, and content semantics documented in Section 4 serve as a forensic reference that guides examiners to relevant traces without prior knowledge of the application.

6.2 Limitations of This Study

During our analysis period, the OpenClaw codebase evolved rapidly. While this is a general phenomenon in digital forensics [27], the fast-paced development style rooted in "vibe coding practices" [19] and the increasing autonomy of machines creating software aggravates the situation and makes OpenClaw a fast moving target for empirical study. Accordingly, we restricted our examination to persistent on-disk artifacts and excluded main memory and network traffic. We also did not assess the durability of this evidence over extended periods. Moreover, we focused on core functionality using a set of representative actions and did not consider companion devices. Our experiments were conducted on a single operating system (Debian Linux). While the application-layer artifacts are platform-independent (cf. Section 3), OS-level traces such as file system metadata semantics, journal entries, or package-manager records will differ across platforms and warrant dedicated cross-platform validation. The LLM's inherent nondeterminism complicated reproducible experiments: despite fixing all variables under our control, we could not make OpenClaw behave completely identically across runs. This highlights that agentic systems complicate reproducible analysis compared to traditional, non-agentic software. The proposed taxonomy is inductively derived from our analysis of OpenClaw. A preliminary literature-based cross-mapping against

published artifact inventories from two additional studies supports the taxonomy's structural completeness (Table 2), but hands-on forensic analysis of additional agentic systems is needed to fully validate its generalizability.

6.3 The Need for an Agentic AI Investigation Framework

Nondeterminism. Our analysis revealed that repeated execution of identical actions with fixed conditions does not yield identical traces. For example, issuing a reminder request may either create a cron job (in `~/.openclaw/cron/`) or instead modify `HEARTBEAT.md`, depending on the context envelope provided to the LLM. We distinguish three sources of nondeterminism: (1) sampling stochasticity: the LLM produces different reasoning chains and tool selections for the same prompt; (2) context assembly variability: the dynamically constructed context window incorporating evolving memory files and workspace content differs subtly between runs; and (3) timing-dependent external state: responses from third-party services are inherently non-reproducible. Crucially, not all artifacts are equally affected. Structural artifacts, such as configuration files, credential stores, directory layouts, and file creation patterns, remained deterministic across our runs, while LLM-generated content, i.e., reasoning traces, tool-choice sequences, and agent-authored memory entries, exhibited variation.

Abstraction. These observations suggest two qualitatively different classes of traces: (i) conventional, deterministic artifacts produced by rule-based software components, and (ii) agent-generated artifacts whose creation depends on LLM-mediated planning and tool choice. The latter class has important implications for reconstruction and attribution, because the same high-level intent can be realized through different execution paths and, consequently, different local traces, which can be conceptualized as an additional abstraction layer [4] in trace creation.

Challenges in Context Reconstruction. Our experiments showed that it is highly relevant to determine which context was available to the LLM at what point in time. Since the workspace environment, memory, and capabilities naturally evolve and change over time, this is difficult to determine. The correlation of modified timestamps of the context-providing files in the workspace or memory directory might give an indication but cannot provide the full picture either since many relevant parameters are difficult to determine retrospectively. A practical way to reduce this interpretive uncertainty is to recover as many parameters as possible that constrain the execution environment (e.g., available tools and permissions). In practice, however, some of these parameters are difficult to determine retrospectively; for instance, capability availability is not logged explicitly.

Closely related are difficulties in reconstructing the agent's intent. While full access to the underlying reasoning process has been shown to provide supporting details for understanding, interpreting, and validating outputs in digital forensics tasks [21], the trend over the course of 2025 toward encrypting the internal thought process and exposing only model-generated summaries[6] limits what is available in practice. Moreover, the visible summaries that remain cannot necessarily be treated as fully faithful records of the underlying reasoning, as providers' own evaluations and independent analyses have shown that reasoning models frequently fail to verbalize the cues that actually shaped their answers [2, 5].

Generally, we see the need for clear guidance on the task of context reconstruction; our taxonomy proposal in Section 5 may provide a viable foundation, but it still requires further development.

7 Future Work

Future work includes extending the analysis to additional AI assistants and deployment settings with different communication channels and OSes. We further plan to broaden the set of covered trace classes (e.g., main

[6]See https://docs.claude.com/en/docs/build-with-claude/extended-thinking, https://platform.openai.com/docs/guides/reasoning, https://cookbook.openai.com/examples/responses_api/reasoning_items, https://ai.google.dev/gemini-api/docs/thinking#signatures, all accessed on 15.07.2026.

memory and network-related artifacts) and to apply and potentially refine the taxonomy and workflow across platforms, diverse configurations and usage scenarios. Reconstructing model reasoning from encrypted blobs remains methodologically open. Future work should explore model-assisted decoding of thought signatures and inference from residual traces across the Configuration, Knowledge, Communication, and Action planes. More fundamentally, we identify risks to digital forensic capabilities arising from (i) the accelerated pace of software production (e.g., via "vibe coding") and (ii) the absence of robust forensic readiness measures, which may hinder the reliable investigation of actions performed by AI agents with full system privileges.

8 Conclusion

Agentic AI systems are becoming a durable part of the software ecosystem. As they increasingly act on behalf of users they will reshape digital processes and become a consequential object of digital investigations. At the same time, their delegated and context-dependent execution challenges long-standing forensic assumptions about reproducibility, provenance, and the completeness of local traces.

In this work, we present, to our knowledge, the first forensic analysis of a widely used personal AI assistant, taking *OpenClaw* as our case study. We combined an architectural perspective with static code analysis and differential forensic analysis to identify recoverable on-disk traces and to relate these traces to agent-relevant investigative questions, for which we provide an analysis tool as open-source software. We further generalized our observations by proposing an artifact taxonomy with five planes: reasoning and cognition, identity and configuration, knowledge and recall, communication and I/O, and actions and effects—to structure investigations of agentic assistants. Our findings also highlight why agentic systems cannot be approached like traditional software. First, agentic execution introduces nondeterminism: the same user-level request may be realized through different tool choices and different state transitions, producing different artifacts. Second, LLMs add an additional layer of abstraction between intent, actions, and observable traces, and third, the relevant context that shaped a decision is difficult to reconstruct retrospectively. Overall, agentic AI shifts the conditions of analysis, compelling a reassessment of long-standing assumptions in a landscape shaped by delegated agency, autonomy, and model-mediated execution, underscoring the need for sustained, systematic forensic research of agentic systems.

Acknowledgments

Jan Gruber was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 570978749.

Use of AI writing assistance

The authors used Claude Opus 4.8 and Gemini 3.1 Pro as writing aids to identify typographical and grammatical issues and to improve the clarity and style of selected passages. All suggestions were critically assessed and, where appropriate, adapted by the authors.

References

[1] Mohamad Abou Ali, Fadi Dornaika, and Jinan Charafeddine. 2026. Agentic AI: a comprehensive survey of architectures, applications, and future directions. *Artif. Intell. Rev.* 59, 1 (2026), 11. https://doi.org/10.1007/S10462-025-11422-4

[2] Iván Arcuschin, Jett Janiak, Robert Krzyzanowski, Senthooran Rajamanoharan, Neel Nanda, and Arthur Conmy. 2025. Chain-of-Thought Reasoning In The Wild Is Not Always Faithful. *arXiv preprint arXiv:2503.08679* (March 2025). https://doi.org/10.48550/arXiv.2503.08679 v4, June 2025.

[3] Ibrahim M. Baggili and Vahid Behzadan. 2020. Founding The Domain of AI Forensics. In *Proceedings of the Workshop on Artificial Intelligence Safety, co-located with 34th AAAI Conference on Artificial Intelligence, SafeAI@AAAI 2020, New York City, NY, USA, February 7, 2020 (CEUR Workshop Proceedings, Vol. 2560)*, Huáscar Espinoza, José Hernández-Orallo, Xin Cynthia Chen, Seán S. ÓhÉigeartaigh,

Xiaowei Huang, Mauricio Castillo-Effen, Richard Mallah, and John A. McDermid (Eds.). CEUR-WS.org, 31–35. https://ceur-ws.org/Vol-2560/paper53.pdf

[4] Brian D. Carrier. 2003. Defining Digital Forensic Examination and Analysis Tools Using Abstraction Layers. *Int. J. Digit. EVid.* 1, 4 (2003). http://www.utica.edu/academic/institutes/ecii/publications/articles/A04C3F91-AFBB-FC13-4A2E0F13203BA980.pdf

[5] Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, Arushi Somani, Peter Hase, Misha Wagner, Fabien Roger, Vlad Mikulik, Samuel R. Bowman, Jan Leike, Jared Kaplan, and Ethan Perez. 2025. Reasoning Models Don't Always Say What They Think. *arXiv preprint arXiv:2505.05410* (May 2025). https://doi.org/10.48550/arXiv.2505.05410 Anthropic Alignment Science Team. Companion blog post: "Reasoning models don't always say what they think," Anthropic, 3 April 2025, https://www.anthropic.com/research/reasoning-models-dont-say-think.

[6] Maxim Chernyshev, Zubair A. Baig, and Robin Ram Mohan Doss. 2024. Towards Large Language Model (LLM) Forensics Using LLM-based Invocation Log Analysis. In *Proceedings of the 1st ACM Workshop on Large AI Systems and Models with Privacy and Safety Analysis, LAMPS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, Bo Li, Wenyuan Xu, Jieshan Chen, Yang Zhang, Jason Xue, Shuo Wang, Guangdong Bai, and Xingliang Yuan (Eds.). ACM, 89–96. https://doi.org/10.1145/3689217.3690616

[7] Hyunji Chung, Jungheum Park, and Sangjin Lee. 2017. Digital forensic approaches for Amazon Alexa ecosystem. *Digit. Investig.* 22 Supplement (2017), S15–S25. https://doi.org/10.1016/J.DIIN.2017.06.010

[8] Jona Crasselt and Gaston Pugliese. 2024. Started Off Local, Now We're in the Cloud: Forensic Examination of the Amazon Echo Show 15 Smart Display. In *Proceedings of the Digital Forensics Research Conference Europe (DFRWS USA)*, Parag Rughani and Frank Breitinger (Eds.). dfrws.org, Baton Rouge, Louisiana, 1–11. https://doi.org/10.48550/ARXIV.2408.15768

[9] Evangelos Dragonas, Costas Lambrinoudakis, and Panagiotis Nakoutis. 2024. Forensic analysis of OpenAI's ChatGPT mobile application. *Forensic Sci. Int. Digit. Investig.* 50 (2024), 301801. https://doi.org/10.1016/J.FSIDI.2024.301801

[10] Johannes Fähndrich, Wilfried Honekamp, Roman Povalej, Heiko Rittelmeier, Silvio Berner, and Dirk Labudde. 2023. Digital forensics and strong AI: A structured literature review. *Forensic Sci. Int. Digit. Investig.* 46 (2023), 301617. https://doi.org/10.1016/J.FSIDI.2023.301617

[11] Matija Franklin, Nenad Tomašev, Julian Jacobs, Joel Z. Leibo, and Simon Osindero. 2026. AI Agent Traps. *SSRN Electronic Journal* (2026). https://doi.org/10.2139/ssrn.6372438

[12] Simson L. Garfinkel. 2009. Automating Disk Forensic Processing with SleuthKit, XML and Python. In *Fourth International IEEE Workshop on Systematic Approaches to Digital Forensic Engineering, SADFE 2009, Berkeley, California, USA, May 21, 2009*. IEEE Computer Society, 73–84. https://doi.org/10.1109/SADFE.2009.12

[13] Simson L. Garfinkel. 2012. Digital forensics XML and the DFXML toolset. *Digit. Investig.* 8, 3-4 (2012), 161–174. https://doi.org/10.1016/J.DIIN.2011.11.002

[14] Simson L. Garfinkel, Alex J. Nelson, and Joel Young. 2012. A general strategy for differential forensic analysis. *Digit. Investig.* 9, Supplement (2012), S50–S59. https://doi.org/10.1016/J.DIIN.2012.05.003

[15] Ken Huang and Jerry Huang. 2025. *AI Agent Tools and Frameworks*. Springer Nature Switzerland, 23–50. https://doi.org/10.1007/978-3-031-90026-6_2

[16] Ken Huang and Chris Hughes. 2025. *Introduction to Agentic AI: Foundations, Drivers, and Risks*. Springer Nature Switzerland, 3–16. https://doi.org/10.1007/978-3-032-02130-4_1

[17] Sven Kälber, Andreas Dewald, and Felix C. Freiling. 2013. Forensic Application-Fingerprinting Based on File System Metadata. In *Seventh International Conference on IT Security Incident Management and IT Forensics, IMF 2013, Nuremberg, Germany, March 12-14, 2013*, Holger Morgenstern, Ralf Ehlert, Felix C. Freiling, Sandra Frings, Oliver Göbel, Detlef Günther, Stefan Kiltz, Jens Nedon, and Dirk Schadt (Eds.). IEEE Computer Society, 98–112. https://doi.org/10.1109/IMF.2013.20

[18] Sahaya Jestus Lazer, Kshitiz Aryal, Maanak Gupta, and Elisa Bertino. 2026. A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-case Prototypes. arXiv:2601.05293 [cs.CR] https://arxiv.org/abs/2601.05293

[19] Christian Meske, Tobias Hermanns, Esther von der Weiden, Kai-Uwe Loser, and Thorsten Berger. 2025. Vibe Coding as a Reconfiguration of Intent Mediation in Software Development: Definition, Implications, and Research Agenda. *IEEE Access* 13 (2025), 213242–213259. https://doi.org/10.1109/ACCESS.2025.3645466

[20] Joachim Metz. 2025. Digital Forensics Artifacts Repository Documentation. https://github.com/ForensicArtifacts/artifacts. https://artifacts.readthedocs.io/en/latest/ Accessed: 2026-02-03.

[21] Gaëtan Michelet, Janine Schneider, Aruna Withanage, and Frank Breitinger. 2026. Hey GPT-OSS, looks like you got it – Now walk me through it! An assessment of the reasoning language models chain of thought process for digital forensics. *Forensic Science International: Digital Investigation* 56 (2026), 302052. https://doi.org/10.1016/j.fsidi.2026.302052 DFRWS EU 2026 - Selected Papers from the 13th Annual Digital Forensics Research Conference Europe.

[22] Ashis Kumar Pati. 2025. Agentic AI: A Comprehensive Survey of Technologies, Applications, and Societal Implications. *IEEE Access* 13 (2025), 151824–151837. https://doi.org/10.1109/ACCESS.2025.3585609

[23] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. 2026. AI Agents vs. Agentic AI: A Conceptual taxonomy, applications and challenges. *Inf. Fusion* 126 (2026), 103599. https://doi.org/10.1016/J.INFFUS.2025.103599

[24] Mark Scanlon, Frank Breitinger, Christopher Hargreaves, Jan-Niclas Hilgert, and John Sheppard. 2023. ChatGPT for digital forensic investigation: The good, the bad, and the unknown. *Forensic Science International: Digital Investigation* 46 (2023), 301609. https://doi.org/10.1016/j.fsidi.2023.301609

[25] Johannes Schneider and Frank Breitinger. 2023. Towards AI forensics: Did the artificial intelligence system do it? *J. Inf. Secur. Appl.* 76 (2023), 103517. https://doi.org/10.1016/J.JISA.2023.103517

[26] Natalie Shapira, Chris Wendler, Avery Yen, Gabriele Sarti, Koyena Pal, Olivia Floody, Adam Belfki, Alexander R. Loftus, Aditya Ratan Jannali, Nikhil Prakash, Jasmine Cui, Giordano Rogers, Jannik Brinkmann, Can Rager, Amir Zur, Michael Ripa, Aruna Sankaranarayanan, David Atkinson, Rohit Gandikota, Jaden Fiotto-Kaufman, EunJeong Hwang, Hadas Orgad, P. Sam Sahil, Negev Taglicht, Tomer Shabtay, Atai Ambus, Nitay Alon, Shiri Oron, Ayelet Gordon-Tapiero, Yotam Kaplan, Vered Shwartz, Tamar Rott Shaham, Christoph Riedl, Reuth Mirsky, Maarten Sap, David Manheim, Tomer Ullman, and David Bau. 2026. Agents of Chaos. *CoRR* abs/2602.20021 (2026). https://doi.org/10.48550/ARXIV.2602.20021 arXiv:2602.20021

[27] Hannes Spichiger and Frank Adelstein. 2025. Preserving meaning of evidence from evolving systems. *Digit. Investig.* 52, Supplement (2025), 301867. https://doi.org/10.1016/J.FSIDI.2025.301867

[28] Clinton Walker, Taha Gharaibeh, Ruba Alsmadi, Cory Lloyd Hall, and Ibrahim M. Baggili. 2024. Forensic Analysis of Artifacts from Microsoft's Multi-Agent LLM Platform AutoGen. In *Proceedings of the 19th International Conference on Availability, Reliability and Security, ARES 2024, Vienna, Austria, 30 July 2024 - 2 August 2024*. ACM, 198:1–198:9. https://doi.org/10.1145/3664476.3670908

[29] Akila Wickramasekara and Mark Scanlon. 2024. A Framework for Integrated Digital Forensic Investigation Employing AutoGen AI Agents. In *2024 12th International Symposium on Digital Forensics and Security (ISDFS)*. 01–06. https://doi.org/10.1109/ISDFS60797.2024.10527235

A Supplementary Material

We provide a supplementary repository alongside this paper, containing the machine-readable description of relevant forensic artifacts, a Python-based tool to support the forensic analysis of OpenClaw, and documentation of the analysis process in the form of the `DFXML` outputs for our experiments.

A.1 Artifact Examiner Tool

To facilitate the analysis of OpenClaw, we developed the **`artifact-examiner`**, a Python-based command-line interface (CLI) tool included in the `artifact-examiner/` directory. This tool leverages our analysis results to automatically ingest, parse, and correlate forensic data from an OpenClaw installation.

The Artifact Examiner offers several capabilities for investigators:

- **Unified Timeline Analysis**: It reconstructs a chronological view of all agent activities by merging events from logs, session transcripts, and configuration changes. This allows analysts to trace the sequence of actions leading up to a specific event.
- **Session Inspection**: The tool provides interactive browsers for exploring agent sessions, including the ability to reconstruct conversation flows, tool invocations, and subagent spawns.
- **Anti-Forensics Detection**: By comparing execution logs with persistent session records, the tool can identify discrepancies that may indicate deleted sessions or tampered data, highlighting potential anti-forensic activities.
- **Capability Analysis**: It tracks the evolution of the agent's capabilities over time, such as changes in available tools, models, and permissions, providing insight into the agent's potential impact on the system.

Refer to the repository for a comprehensive listing of its capabilities. During our own analyses, we noted that the `artifact-examiner` reduces the manual effort required to analyze human interaction and agent behavior, while ensuring a consistent and reproducible investigation process.

A.2 Forensic Artifact Definition

We provide a technical and concise description of forensic artifacts via the `artifacts/data/openclaw.yml` file following the specification of the *Digital Forensics Artifacts Repository* [20]. This YAML file serves as a machine-readable schema that defines the location, structure, and semantic meaning of important artifacts generated by the OpenClaw agent. Each artifact entry includes:

- **Name**: A unique identifier for the artifact (e.g., `LinuxOpenClawSessionTranscriptJsonl`, `LinuxOpenClawDailyJsonLogFiles`).
- **Documentation**: A brief description of the artifact's purpose, content, and forensic significance. This documentation is embedded directly within the schema to ensure it remains synchronized with the implementation.
- **Sources**: The file paths or glob patterns where the artifact can be found on the target system (e.g., `~/.openclaw/agents/<agentId>/sessions/*.jsonl`).

We aim to enable both human analysts and automated tools to reliably locate and interpret forensic data of OpenClaw.

A.3 Experimental Methodology

The artifact documentation within the `artifacts/` directory was informed through a controlled series of experiments designed to isolate file system changes associated with specific user and agent actions. The methodology involved the following steps:

(1) **Baseline Establishment**: A clean virtual machine (VM) snapshot was created with the base operating system and necessary dependencies but without the OpenClaw agent installed.
(2) **Action Execution**: Specific actions were performed in a sequential manner, such as installing the agent, onboarding, sending text and voice messages, adding calendar events, and spawning subagents.
(3) **Differential Analysis**: After each action, the VM state was captured, and a differential analysis was performed against the previous state to identify created, modified, or deleted files.

A.4 Availability

The supplementary material, including the analysis tool `artifact-examiner`, the `openclaw.yml` schema, and the differential analysis, is published at:

https://github.com/jgru/forensic-analysis-of-openclaw

Artifacts have been redacted where necessary to remove sensitive personal information.